

AI-Powered Unified Vulnerability Intelligence Platform — Security Engine

Overview

The Security Engine is the core analytical component of the platform. It is responsible for ingesting source code from GitHub repositories, executing multi-layer security scans in isolated environments, and normalizing diverse tool outputs into a single, actionable intelligence report.

This component handles the primary heavy lifting of the project, ensuring that untrusted code is scanned safely without impacting the host system.

Core Features

- Multi-Layer Analysis: Simultaneous scanning of Application (SAST), Dependency (SCA), Secrets, and Infrastructure (Dockerfile) layers.
- Docker Isolation: Every scan runs inside an ephemeral, hardened container to prevent side-effects on the host machine.
- Unified Schema: Converts disparate JSON formats from industry-standard tools into a consistent "Intelligence Schema" used by the Risk Modeling and AI components.
- Language Agnostic: Tested and verified on Node.js, Python, and multi-framework repositories.
- Parallel Execution: Uses Python multi-threading to run security tools concurrently, significantly reducing total scan time.

Architecture

The engine follows a modular design:

1. Trigger: Receives a GitHub URL via the `orchestrate_scan()` interface.
2. Setup: Clones the repository into a unique, temporary workspace.
3. Execution: Dispatches parallel Docker containers for Opengrep, Trivy, Gitleaks, and Hadolint.
4. Normalization: Aggregates raw outputs and maps them to standard CVSS and Severity levels.
5. Output: Generates a single, time-stamped JSON report.
6. Cleanup: Purges temporary source code to maintain system integrity and storage efficiency.

Quick Start for Team Members

1. Prerequisites

- Docker Desktop (Running)
- Python 3.11+

2. Build the Engine

From the project root, build the core image:

```
docker build -t security-engine:latest -f security-engine.Dockerfile .
```

3. Run a Test Scan

```
python engine.py https://github.com/OWASP/NodeGoat.git
```

Team Integration Contracts

- For Platform Lead (Bhavya): See README_ENGINE.md for programmatic integration details.
- For Risk Lead (Bhumika): The engine provides standardized cvss (float) and severity (string) for all findings.
- For AI Lead (Divyanshi): The engine provides evidence (code snippets) and fix suggestions for LLM processing.

Unified Vulnerability Intelligence — Security Engine

This module provides the core scanning and normalization capabilities for the platform. It is designed to be language-agnostic and executes multiple security engines in parallel using isolated Docker containers.

Integrated Engines

- **Opengrep:** Application Layer (SAST)
- **Trivy:** Dependency Layer (SCA)
- **Gitleaks:** Secret Detection Layer
- **Hadolint:** Infrastructure/Dockerfile Layer

Integration Guide (For Platform & Backend Lead)

The engine is designed as a "Black Box" module. You can trigger it from your Node.js backend using a child process or by importing it into a Python-based Redis worker.

Standard Execution (CLI)

```
python engine.py <github_repo_url>
```

Programmatic Integration (Python)

```
from engine import orchestrate_scan
```

```
# Returns a dictionary containing the full normalized report
```

```
report = orchestrate_scan("https://github.com/OWASP/NodeGoat.git")

print(report['scan_id'])

print(f"Total Issues: {report['total_findings']}")
```

Unified intelligence Schema

Every finding returned by the engine follows this strict schema to ensure compatibility with the Risk Engine and AI Security Agent:

Field	Type	Description
<code>id</code>	String	Unique identifier for the vulnerability (e.g., CVE-XXX or Rule-ID)
<code>tool</code>	String	The source tool (<code>opengrep</code> , <code>trivy</code> , <code>gitleaks</code> , <code>hadolint</code>)
<code>category</code>	String	High-level layer (<code>sast</code> , <code>sca</code> , <code>secrets</code> , <code>infra</code>)
<code>title</code>	String	Concise title of the security issue
<code>description</code>	String	Detailed technical explanation
<code>severity</code>	String	Standardized level: <code>CRITICAL</code> , <code>HIGH</code> , <code>MEDIUM</code> , <code>LOW</code> , <code>INFO</code>
<code>cvss</code>	Float	0.0 - 10.0 risk score (uses tool data or heuristic fallback)
<code>file</code>	String	Path to the affected file
<code>line</code>	Integer	Line number (0 if file-wide)
<code>evidence</code>	String	The raw code snippet or package name causing the issue
<code>fix</code>	String	Remediation guidance

Field	Type	Description
<code>references</code>	Array	List of URLs for further reading/CVE details

Requirements

- **Docker Desktop:** Must be running and accessible to the user executing the script.
- **Image:** `security-engine:latest` must be built (see `security-engine.Dockerfile`).
- **Python 3.11+**